

Section 1: Basic Operations (Easy)

These problems build your foundation — traversal, insertion, deletion, and basic manipulation.

Q1. Reverse a Linked List Reverse a singly linked list both iteratively and recursively. 

LeetCode: <https://leetcode.com/problems/reverse-linked-list/>

Q2. Middle of the Linked List Find the middle node of a linked list. If two middle nodes exist, return the second one.  LeetCode: <https://leetcode.com/problems/middle-of-the-linked-list/>

Q3. Merge Two Sorted Lists Merge two sorted linked lists and return the merged list in sorted order.  LeetCode: <https://leetcode.com/problems/merge-two-sorted-lists/>

Q4. Delete Node in a Linked List Delete a node given only access to that node (not the head).

 LeetCode: <https://leetcode.com/problems/delete-node-in-a-linked-list/>

Q5. Remove Duplicates from Sorted List Given a sorted linked list, delete all duplicates so each element appears only once.  LeetCode:

<https://leetcode.com/problems/remove-duplicates-from-sorted-list/>

Section 2: Two Pointer / Slow-Fast Pointer (Easy-Medium)

These problems rely entirely on the slow and fast pointer technique — one of the most important patterns in linked list problems.

Q1. Linked List Cycle Detection Determine if a linked list has a cycle in it using Floyd's algorithm.  LeetCode: <https://leetcode.com/problems/linked-list-cycle/>

Q2. Linked List Cycle II — Find Entry Point Given a linked list with a cycle, return the node where the cycle begins.  LeetCode: <https://leetcode.com/problems/linked-list-cycle-ii/>

Q3. Remove Nth Node from End of List Remove the nth node from the end of the list in a single pass using two pointers.  LeetCode:

<https://leetcode.com/problems/remove-nth-node-from-end-of-list/>

Q4. Palindrome Linked List Check whether a linked list is a palindrome. Aim for $O(n)$ time and $O(1)$ space.  LeetCode: <https://leetcode.com/problems/palindrome-linked-list/>

Q5. Intersection of Two Linked Lists Find the node at which two singly linked lists intersect. Return null if they do not.  LeetCode:

<https://leetcode.com/problems/intersection-of-two-linked-lists/>

Section 3: Reversal Problems (Medium)

These problems extend basic reversal to partial or grouped reversals — a very common interview category.

Q1. Reverse Linked List II Reverse the nodes of the list from position left to position right in one pass. 🔗 LeetCode: <https://leetcode.com/problems/reverse-linked-list-ii/>

Q2. Reverse Nodes in k-Group Reverse every k adjacent nodes in the linked list. If remaining nodes are fewer than k, leave them as-is. 🔗 LeetCode: <https://leetcode.com/problems/reverse-nodes-in-k-group/>

Q3. Reorder List Reorder a list from $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ to $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$ in-place. 🔗 LeetCode: <https://leetcode.com/problems/reorder-list/>

Q4. Swap Nodes in Pairs Swap every two adjacent nodes in a linked list without modifying node values. 🔗 LeetCode: <https://leetcode.com/problems/swap-nodes-in-pairs/>

Q5. Rotate List Rotate the linked list to the right by k places. 🔗 LeetCode: <https://leetcode.com/problems/rotate-list/>

Section 4: Merge, Sort and Arithmetic (Medium)

These problems combine linked lists with sorting logic and number representation.

Q1. Sort List Sort a linked list in $O(n \log n)$ time and $O(1)$ space using merge sort. 🔗 LeetCode: <https://leetcode.com/problems/sort-list/>

Q2. Merge K Sorted Lists Merge k sorted linked lists into one sorted list. Use a min-heap or divide and conquer. 🔗 LeetCode: <https://leetcode.com/problems/merge-k-sorted-lists/>

Q3. Add Two Numbers Two non-empty linked lists represent two non-negative integers stored in reverse order. Add them and return the sum as a linked list. 🔗 LeetCode: <https://leetcode.com/problems/add-two-numbers/>

Q4. Add Two Numbers II Same as above but numbers are stored in forward order. You cannot reverse the input lists. 🔗 LeetCode: <https://leetcode.com/problems/add-two-numbers-ii/>

Q5. Remove Duplicates from Unsorted List Remove all duplicates from an unsorted linked list. Try both $O(n)$ with extra space and $O(n^2)$ without. 🔗 GFG: <https://www.geeksforgeeks.org/remove-duplicates-from-an-unsorted-linked-list/>

Section 5: Advanced and Hard Problems

These problems are frequently asked in top product-based company interviews and require combining multiple techniques.

Q1. Copy List with Random Pointer Clone a linked list where each node has a next pointer and a random pointer that can point to any node or null.  LeetCode:

<https://leetcode.com/problems/copy-list-with-random-pointer/>

Q2. LRU Cache Design a data structure that follows the Least Recently Used cache eviction policy using a doubly linked list and hashmap.  LeetCode:

<https://leetcode.com/problems/lru-cache/>

Q3. Flatten a Multilevel Doubly Linked List Flatten a doubly linked list where nodes may have a child pointer pointing to a separate doubly linked list.  LeetCode:

<https://leetcode.com/problems/flatten-a-multilevel-doubly-linked-list/>

Q4. Merge Sort on Doubly Linked List Sort a doubly linked list using merge sort without using extra space for node creation.  GFG:

<https://www.geeksforgeeks.org/merge-sort-for-doubly-linked-list/>

Q5. Reverse a Linked List in Groups of Given Size (Variable k) Reverse alternate groups of size k — reverse first k, skip next k, reverse next k, and so on.  GFG:

<https://www.geeksforgeeks.org/reverse-a-list-in-groups-of-given-size/>